# 🗂️ 0. GitHub Repository Architecture & Standards

Before writing any code, set up your GitHub repository to keep your SQL files organized. Do not commit the raw Kaggle CSV files to GitHub; add *.csv and *.bak to your .gitignore file.

## Main Directory Structure

Create this exact folder tree in your main branch before branching off:

```
Plaintext
Olist-Data-Warehouse/
├── .gitignore
├── README.md
├── docs/
│   └── DB2_project.pdf
└── sql/
    ├── 01_staging/
    ├── 02_dw_dimensions/
    ├── 03_dw_facts/
    └── 04_etl_pipelines/
```

## Git Branching Strategy

- **main**: The final, stable code ready for presentation.
- **develop**: The shared integration branch. All Pull Requests (PRs) go here first.
- **Feature Branches**: Temporary branches where you do your actual work. You will create a new branch for each phase.

# 🏁 Phase 1: Ingestion & Staging (Extraction & Cleaning)

**Goal:** Move raw data into SQL Server via the SSMS Wizard and prepare Olist_Staging.

## 👤 Yasin's Task Block

- **Active Branch:** feat/yasin-staging
- **Target Files:** Create these inside sql/01_staging/
  - 01_create_stg_customers.sql
  - 02_create_stg_orders.sql
  - 03_create_stg_order_items.sql
- **Execution:**

1. Run your assigned CSVs (customers, orders, order_items) through the SSMS Import Flat File Wizard to create Olist_Source.
2. Write the SQL scripts above to create the Staging tables (adding a LoadDate column). Do not add Primary Keys or Foreign Keys.
3. Commit the scripts to your branch and push to GitHub.

### 👤 Abolfazl's Task Block

- **Active Branch:** feat/abolfazl-staging
- **Target Files:** Create these inside sql/01_staging/
  - 04_create_stg_sellers.sql
  - 05_create_stg_products.sql
  - 06_create_stg_geolocation.sql
  - 07_create_stg_reviews.sql
- **Execution:**

1. Run your assigned CSVs through the SSMS Import Flat File Wizard into Olist_Source.
2. Write the SQL scripts above to create the Staging tables.
3. Commit the scripts to your branch and push to GitHub.

## 🏗️ Phase 2: Dimensional Modeling (Star Schema Design)

**Goal:** Design and create the data warehouse schemas inside Olist_DW.

### 👤 Yasin's Task Block (Sales Mart)

- **Active Branch:** feat/yasin-sales-dw
- **Data Mart Focus:** Analyze revenue, discounts, and purchasing behavior.
- **Target Files:** Create these inside sql/02_dw_dimensions/ and sql/03_dw_facts/
  - 02_dw_dimensions/01_dim_customer.sql (Include IsCurrent, ValidFrom, ValidTo)
  - 02_dw_dimensions/02_dim_product.sql
  - 02_dw_dimensions/03_dim_date.sql
  - 03_dw_facts/01_fact_sales.sql
- **Execution:** Write the CREATE TABLE scripts with strict INT IDENTITY(1,1) Primary Keys and appropriate Foreign Keys.

### 👤 Abolfazl's Task Block (Logistics Mart)

- **Active Branch:** feat/abolfazl-logistics-dw
- **Data Mart Focus:** Analyze delivery scheduling, delays, and freight costs.
- **Target Files:** Create these inside sql/02_dw_dimensions/ and sql/03_dw_facts/
  - 02_dw_dimensions/04_dim_seller.sql
  - 02_dw_dimensions/05_dim_review.sql
  - 03_dw_facts/02_fact_logistics.sql
- **Execution:** Write the CREATE TABLE scripts mapping to your logistics metrics (Freight Value, Delivery Delay). Coordinate with Yasin to share dim_customer and dim_date.

## ⚙️ Phase 3: ETL & Slowly Changing Dimensions (Pipelines)

**Goal:** Build the Stored Procedures that transform and load data from Olist_Staging into Olist_DW.

## 👤 Yasin's Task Block

- **Active Branch:** feat/yasin-sales-etl
- **Target Files:** Create these inside sql/04_etl_pipelines/
  - 01_load_dim_customer_scd2.sql
  - 02_load_dim_product_scd3.sql
  - 03_load_fact_sales.sql
- **Execution:**

1. Write the T-SQL MERGE statement to handle SCD Type 2 changes for Customer geographic regions.
2. Write the SCD Type 3 logic for Product hierarchy changes.
3. Write the LEFT JOIN Stored Procedure to populate Fact_Sales.

## 👤 Abolfazl's Task Block

- **Active Branch:** feat/abolfazl-logistics-etl
- **Target Files:** Create these inside sql/04_etl_pipelines/
  - 04_load_dim_seller_scd1.sql
  - 05_load_dim_review.sql
  - 06_load_fact_logistics.sql
  - 07_create_columnstore_indexes.sql
- **Execution:**

1. Write the update logic for SCD Type 1 to overwrite Seller contact info without keeping history.
2. Write the Stored Procedure for Fact_Logistics, calculating time delays.
3. Write the script to apply Non-Clustered Columnstore Indexes to the Fact table.

# 🚀 Phase 4: Integration, Merging & Validation

**Goal:** Combine the GitHub branches and validate the entire architecture.

1. **Pull Requests (PR):** Both Yasin and Abolfazl open PRs on GitHub from your feature branches into the develop branch.
2. **Code Review:** Do not approve your own PR. Yasin reviews Abolfazl's SQL; Abolfazl reviews Yasin's SQL. Once approved, merge them into develop.
3. **The Master Execution:** Pull the develop branch to your local machines. Execute the SQL files in order from folders 01 through 04.
4. **Final Dashboards/Queries:** Write SELECT statements against Olist_DW to ensure managers can easily analyze the data without dealing with normalized operational tables.
5. **Final Release:** Merge develop into main. The project is now ready to be submitted.